1. **Write a shell script to print arguments passed to the script in reverse order.**
   Input: sh script.sh 2 5 3
   Output:
   3
   5
   2

```
#!/bin/bash

# get total number of arguments
count=$#

# loop from last argument to first
while [ $count -gt 0 ]
do
   echo ${!count}
   count=$((count-1))
done
```

```
[ec2-user@RedHat scripts]$ bash reverseorder.sh 2 5 3
3
5
2
[ec2-user@RedHat scripts]$
```

2. **Write a script that monitors system performance and sends a mail notification with warning messages about resource usage like ram, disk usage**

Step 1 : Install Required Packages:
First, update the system and install Postfix and required dependencies.

```
sudo yum update -y
sudo yum install postfix postfix-lmdb s-nail cyrus-sasl cyrus-sasl-plain -y
```

Verify installation:

```
rpm -qa | grep postfix
rpm -qa | grep cyrus-sasl
```

- postfix → Mail Transfer Agent used to send emails
- postfix-lmdb → High-performance database backend for Postfix
- s-nail → Command-line mail client
- cyrus-sasl → Provides authentication support
- cyrus-sasl-plain → Enables plain authentication mechanism

Step 2 : Start and Enable Postfix Service
Check Postfix service status:

```
systemctl status postfix
```

If not running, start and enable it:

```
sudo systemctl start postfix
sudo systemctl enable postfix
```

Verify logs:

```
sudo tail -f /var/log/maillog
```

Step 3 : Test Gmail SMTP Connectivity

Install telnet:

```
sudo yum install telnet -y
```

Test SMTP connection:

```
telnet smtp.gmail.com 587
```

Expected output:

```
[ec2-user@RedHat shell_assignment]$ telnet smtp.gmail.com 587
Trying 172.253.115.109...
Connected to smtp.gmail.com.
Escape character is '^]'.
220 smtp.gmail.com ESMTP 6a1803df08f44-8997e7747f0sm101024926d6.51 - gsmtp
```

Step 4 : Backup Original Configuration
Backup Postfix configuration file:

```
sudo cp /etc/postfix/main.cf /etc/postfix/main.cf.backup
ls -l /etc/postfix/main.cf*
```

Step 5 : Create Gmail Authentication File

Create credential file:

```
sudo vi /etc/postfix/sasl_passwd
```

Add your Gmail credentials:

```
[smtp.gmail.com]:587 your-email@gmail.com:your_app_password
```

Set permissions:

```
sudo chmod 600 /etc/postfix/sasl_passwd
```

Create database file:

```
sudo postmap lmdb:/etc/postfix/sasl_passwd
```

Step 6 : Configure Postfix Main Configuration

Edit main configuration file:

```
sudo vi /etc/postfix/main.cf
```

Add the following lines:

```
relayhost = [smtp.gmail.com]:587
smtp_use_tls = yes
smtp_sasl_auth_enable = yes
smtp_sasl_security_options = noanonymous
smtp_sasl_password_maps = lmdb:/etc/postfix/sasl_passwd
smtp_tls_CAfile = /etc/pki/tls/certs/ca-bundle.crt
```

- relayhost → Specifies Gmail SMTP server
- smtp_use_tls → Enables encryption
- smtp_sasl_auth_enable → Enables authentication
- smtp_sasl_password_maps → Specifies credential file
- smtp_tls_CAfile → Specifies trusted certificate authorities

Step 7: Restart and Enable Postfix

Restart and Enable service:

```
sudo systemctl restart postfix
sudo systemctl enable postfix
```

Check status:

```
sudo systemctl status postfix
```

Verify configuration:

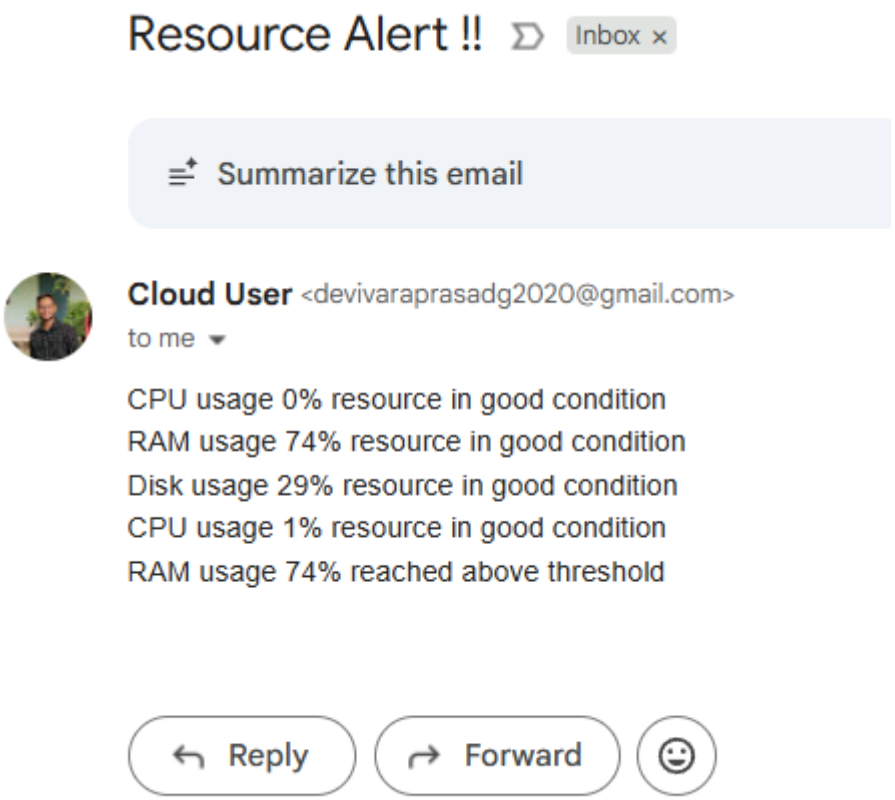
```
sudo postconf -n
```

```
#!/bin/bash

cpu_ug=$(mpstat 1 1 | awk '/Average:/ {printf("%.0f", 100 - $NF)}')
mem_ug=$(free | awk '/Mem/ {printf("%.0f", $3/$2 * 100)}')
dsk_ug=$(df / | awk 'NR==2 {print $5}' | sed 's/%//g')

logfile="/home/ec2-user/resources_alert.log"
mailid="devivaraprasadg2020@gmail.com"

for res_typ in CPU RAM Disk
do
	case $res_typ in
		CPU) res=$cpu_ug;;
		RAM) res=$mem_ug;;
		Disk) res=$dsk_ug;;
	esac

	if (($res >= 60))
	then
		echo "$res_typ usage $res% reached above threshold" >> $logfile
		echo "$res_typ usage $res% reached above threshold"
		tail -n 10 $logfile | mail -s "Resource Alert !!" "$mailid"
	else
		echo "$res_typ usage $res% resource in good condition" >> $logfile
		echo "$res_typ usage $res% resource in good condition"
	fi
done
```

Output:-

```
[ec2-user@RedHat shell_assignment]$ sudo bash system_performance.sh
CPU usage 1% resource in good condition
RAM usage 74% reached above threshold
Disk usage 29% resource in good condition
```

Mail received on reached above threshold:



3. **Write a script to find the files in a directory that were last modified more than 4 weeks ago[For Practise use Mins/Days] and delete them, starting with the ones that consume the most space.**

```
#!/bin/bash

DIR="/home/ec2-user/shell_assignment/logs"

# find files older than 28 days, sort by size, and delete
find "$DIR" -type f -mtime +28 -exec ls -lh {} + | sort -k5 -hr | awk '{print $9}' |
while read file
do
   echo "Deleting $file"
   rm "$file"
done
```

```
[ec2-user@RedHat shell_assignment]$ bash del_4weeks_ago_files.sh
Deleting /home/ec2-user/shell_assignment/logs/fil4.log
Deleting /home/ec2-user/shell_assignment/logs/fil1.log
Deleting /home/ec2-user/shell_assignment/logs/fil2.log
Deleting /home/ec2-user/shell_assignment/logs/fil3.log
```

4. **Explain the difference between running a script with './script' and running it with 'nohup ./script &'.**

#!/bin/bash

echo "Script started"

sleep 30

echo "Script finished"

1. **Running a script using ./script :**
   - Executes script in foreground
   - Terminal remains busy until execution completes
   - Script stops if terminal is closed

```
[ec2-user@RedHat scripts]$ ./forground.sh
Script started
```

2. **Running a script using nohup ./script & :**
   - Executes script in background
   - Terminal remains free for other commands
   - Script continues running even if terminal is closed
   - Output is stored in nohup.out file

```
[ec2-user@RedHat scripts]$ nohup ./forground.sh &
[2] 13798
[ec2-user@RedHat scripts]$ nohup: ignoring input and appending output to 'nohup.out'
```

Output of nohup.out file :-

```
[ec2-user@RedHat scripts]$ cat nohup.out
Script started
Script finished
[ec2-user@RedHat scripts]$
```

5. **Read an integer 'n' and generate the following**

pattern:
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
up to 'n' rows.

```
#!/bin/bash

echo "Enter a number:"
read n

for ((i=1; i<=n; i++))
do
   for ((j=1; j<=i; j++))
   do
      echo -n "$j "
   done
   echo
done
```

```
[ec2-user@RedHat scripts]$ bash pattern.sh
Enter a number:
5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

6. **Write a script to display the contents of the file in reverse order without using tac command**

```
#!/bin/bash

echo "Enter filename:"
read file

lines=$(wc -l < "$file")

while [ $lines -gt 0 ]
do
   sed -n "${lines}p" "$file"
   lines=$((lines-1))
done
```

```
[ec2-user@RedHat scripts]$ bash linesinreverseorder.sh
Enter filename:
reversefile.txt
five
four
three
two
one
```

7. **Write a script to reverse a string using while loop**
   Input: hello
   Output: olleh

```
#!/bin/bash

echo "Enter a string:"
read str

len=${#str}
rev=""

while [ $len -gt 0 ]
do
   char=${str:$((len-1)):1}
   rev="$rev$char"
   len=$((len-1))
done

echo "Reversed string: $rev"
```

```
[ec2-user@RedHat scripts]$ bash revstringwhile.sh
Enter a string:
hello
Reversed string: olleh
[ec2-user@RedHat scripts]$
```

8. **Read an integer 'n' and generate the following pattern:**
   1
   2 3
   4 5 6
   7 8 9 10
   up to 'n' rows."

```
#!/bin/bash

echo "Enter number of rows:"
read n

num=1
```

```
for ((i=1; i<=n; i++))
do
   for ((j=1; j<=i; j++))
   do
      echo -n "$num "
      num=$((num+1))
   done
   echo
done
```

```
[ec2-user@RedHat scripts]$ bash pattern2.sh
Enter number of rows:
4
1
2 3
4 5 6
7 8 9 10
[ec2-user@RedHat scripts]$
```

9. **Write a script that renames a file or directory by replacing its letters with lowercase from uppercase letters.**
   Example: TEst.txt → test.txt

```
#!/bin/bash

echo "Enter file or directory name:"
read name

lower=$(echo "$name" | tr 'A-Z' 'a-z')

mv "$name" "$lower"

echo "Renamed to: $lower"
```

```
[ec2-user@RedHat scripts]$ bash renamecl.sh
Enter file or directory name:
TeXt.txt
Renamed to: text.txt
[ec2-user@RedHat scripts]$
```

10. Write a script that takes multiple inputs from the user as arguments and only prints the arguments if they are integers
Input: sh script.sh abc 2 t5 4 efg ghi 34 67 1 2
Output
The integer arguments are:
2
4
34
67
1
2

```
#!/bin/bash

echo "The integer arguments are:"

for arg in "$@"
do
   if [[ $arg =~ ^[0-9]+$ ]]
   then
      echo "$arg"
   fi
done
```

```
[ec2-user@RedHat scripts]$ bash printonlyintegers.sh abc 2 t5 4 efg ghi 34 67 1 2
The integer arguments are:
2
4
34
67
1
2
[ec2-user@RedHat scripts]$
```